Federated Elections with Encrypted Local Tallies and Verifiable On-Chain Aggregation

Vincenzo Imperati[1], Lorenzo Camilli[1], and Raffaele Ruggeri[1]

Sapienza University of Rome, Italy
imperati@di.uniroma1.it, camilli.1845956@studenti.uniroma1.it,
ruggeri.1934646@studenti.uniroma1.it

Abstract. We study federated elections in which votes remain secret and are counted locally, but the global result must be derived from all local tallies without trusting a central coordinator or exposing local outcomes. Cleartext publication makes inclusion auditable but sacrifices local privacy, while commit-and-reveal only delays disclosure. We present a blockchain-assisted protocol in which each authority submits an *encrypted* local tally vector, a Groth16 proof that the hidden tally is well formed and sums to the public local electorate size, and a signature binding the submission to the election transcript. The contract verifies proofs, records one submission per authority, and maintains the homomorphic aggregate on-chain. Separate proof checks cover trustee setup and threshold opening, so only the global tally is recovered. We provide a reproducible artifact with a Solidity contract, Circom circuits, and an end-to-end replay of accepted submissions, aggregation, and final tally recovery without revealing local tallies.

Keywords: Federated elections · Tally integrity · Zero-knowledge proofs · Homomorphic aggregation

1 Introduction

We study federated elections in which authorized local authorities conduct secret voting and compute local tallies off-chain. Ballot secrecy is already enforced locally and local committees are assumed to count correctly. The integrity gap in this federated workflow lies in the transition from local results to the global result: a central coordinator can omit unfavorable tallies, misreport the aggregate, or exploit early knowledge of partial results.

Publishing local tallies on a blockchain makes inclusion auditable but exposes local outcomes, which can create retaliation, reputational pressure, or strategic bargaining, especially in small electorates. Commit-and-reveal only postpones that disclosure and still leaves an opportunity to abort at the opening stage.

DeCifris Template – Contribution

This privacy and integrity tension motivates the central design choice of this paper: *the local tally should remain encrypted throughout the protocol*. Each authority publishes an encrypted tally vector and a zero-knowledge proof that the hidden tally is structurally valid and sums to its public electorate size. The contract accepts one such submission per authority, fixes the transcript on-chain, computes the homomorphic aggregate, and only the global tally is decrypted.

Our design is closer to open-audit bulletin board systems such as Helios [1] and to multi-authority election protocols [3] than to a ballot-level remote voting stack, but adds three elements for private local tallies: validity proofs for encrypted local results, an on-chain aggregate updated from accepted submissions, and a proof-checked threshold opening transcript that reveals only the final aggregate.

We instantiate encryption with curve-based additive ElGamal, which aligns the homomorphic layer with Circom/Groth16 arithmetic. The work remains at the local-tally level because the problem addressed here is final aggregation from trusted local counts, not ballot-level remote voting.

2 System Model and Attacks

The protocol has four roles. The *owner* initializes the election and the authority registry. Each *local authority* computes its local tally off-chain and submits one encrypted tally vector. The *smart contract* acts as the public bulletin board and proof gate. A small set of *trustees* performs threshold decryption of the final aggregate.

Our assumptions are deliberately narrow. Ballots remain secret inside each authority, local counting is outside the protocol boundary, electorate sizes are known to the contract, and the final result must be derived from the full set of accepted local tallies rather than from a coordinator's private view. We add one privacy requirement that cleartext bulletin boards do not satisfy: local tallies should remain confidential even after the election.

Within this model, we consider five threats: *omission and replacement* by a coordinator; *strategic observation* before the election closes; *local retaliation* under cleartext publication; malformed encrypted submissions that hide negative, oversized, or wrong-sum tallies; and opening inconsistencies in the trustee registry or decryption transcript.

The protocol addresses these threats with encrypted submissions, transcript inclusion, and a finalization rule that cancels incomplete elections: local tallies are never revealed, only the final aggregate is opened, malformed submissions are excluded through zero-knowledge proofs, and trustee setup and opening are proof-checked.

The protocol has explicit limitations. A proof of *well-formedness* is not a proof that the encrypted tally is the *true* local tally, so a fully corrupted local committee remains outside scope. The artifact also uses a dealer-generated threshold key rather

than distributed key generation, and final tally decoding after threshold opening is performed off-chain through bounded discrete-log recovery.

3 Protocol Overview

Setup. The contract records the authorized local authorities, their electorate sizes, an election identifier, a trustee set, a decryption threshold, and a public key for additively homomorphic ElGamal-style encryption [4]. Trustee public shares are derived from threshold secret shares; the artifact instantiates these shares using Shamir secret sharing [7]. Before the election starts, the owner submits a Groth16 proof that the registered trustee public shares are consistent with the election public key and threshold.

Encrypted local tally. Authority A_i computes off-chain counts $(t_{i,1}, \ldots, t_{i,m})$. Instead of publishing the counts, the authority encrypts each component and obtains a ciphertext vector $C_i = (\mathsf{Enc}(t_{i,1}), \ldots, \mathsf{Enc}(t_{i,m}))$. Because the encryption is additively homomorphic, component-wise aggregation of ciphertexts corresponds to component-wise addition of the hidden tallies.

Zero-knowledge validity proof. Together with C_i, the authority submits a Groth16 proof π_i [6] for the statement that there exist plaintext counts and encryption randomness such that the published ciphertexts encrypt those counts, each count is in range, and the sum of the counts equals the public electorate size of A_i. The authority also signs the transcript, binding the election identifier, authority address, electorate size, and ciphertext coordinates to the submitting authority.

On-chain acceptance. The contract accepts exactly one encrypted tally per authorized authority. A submission is accepted only if it arrives before the deadline, has not already been submitted by that authority, matches the expected public inputs, and passes on-chain Groth16 verification. Once accepted, the ciphertext vector is fixed in the transcript and folded into an aggregate ciphertext maintained by BabyJub point addition.

Deterministic aggregation. No privileged actor posts the aggregate. The contract maintains the component-wise homomorphic sum of accepted ciphertexts, and any observer can recompute the same value from the accepted transcript.

Threshold decryption. Only the aggregate ciphertext is opened. The design follows threshold decryption for voting protocols [5]. For each candidate component, trustees publish a share point together with a Groth16 proof that the share is consistent with both the trustee's public share and the corresponding contract-maintained ciphertext. The contract accepts only verified shares. Once a threshold

of valid shares is available, any observer can reconstruct the final tally and determine the winner. The artifact performs the final bounded discrete-log decoding step off-chain, while share acceptance is enforced on-chain.

Failure handling. If the submission deadline expires before all authorities have posted an accepted encrypted tally, the election is cancelled instead of producing a partial result. Cancellation preserves the invariant that the final result must come from the complete accepted set of local tallies.

4 Implementation and Artifact

The companion artifact is available in the public repository[1]. The repository contains a Solidity contract for authority registration, on-chain Groth16 verification, encrypted transcript recording, aggregate maintenance, trustee setup, and final opening. Alongside the contract, the artifact includes three Circom circuits [2]: local-tally validity, trustee-registry consistency, and partial decryption share validity.

For reviewers, the entry point is `yarn validate`. The command rebuilds circuit artifacts if needed, runs the contract tests, and executes an end-to-end replay with three authorities, two candidates, a 2-of-3 trustee committee, and a dealer-generated threshold key. The replay generates real proofs, submits signed encrypted local tallies, reconstructs the aggregate from the transcript, compares that value with the on-chain aggregate, and recovers the final tally.

The local tallies are never published in clear. The public transcript contains only accepted ciphertexts, proof and signature checks, trustee setup, verified decryption shares, and the final aggregate after threshold opening.

5 Discussion

Origin. The project originated in the Cryptography track of the 2024 hackathon organized by De Cifris and XRPL Commons. The track's federated-election scenario highlighted a concrete trust gap: local committees already preserve ballot secrecy and count local votes, while a central actor still collects tallies and announces the final result. That scenario led us to focus on federation-wide aggregation integrity rather than replacing local voting.

Approach. The protocol separates local tallying from public aggregation. Local authorities publish signed encrypted tally vectors with validity proofs; the contract accepts one proof-checked submission per authority, maintains the homomorphic aggregate, and verifies the threshold-opening transcript before the global result is recovered.

[1] `https://github.com/VincenzoImp/encrypted-local-tally-integrity`.

Deployment path. The artifact shows a practical route toward institutional governance systems, provided deployment work adds operational key management, audits, authority onboarding, monitoring, and administrator interfaces. The present implementation remains a research prototype, yet demonstrates a reproducible workflow for private local tallies and verifiable global aggregation.

6 Conclusion

We presented a blockchain-assisted protocol for deriving a global result from encrypted local tallies without exposing local outcomes or trusting a central coordinator to compute the aggregate. The accompanying artifact demonstrates the core workflow end to end: proof-checked encrypted submissions, deterministic on-chain aggregation, and threshold opening of the final tally only.

References

1. Adida, B.: Helios: Web-based Open-Audit voting. In: van Oorschot, P.C. (ed.) USENIX Security 2008. pp. 335–348. USENIX Association, San Jose, CA (Jul / Aug 2008), https://www.usenix.org/event/sec08/tech/adida.html
2. Circom developers: Circom 2 documentation. https://docs.circom.io (2022)
3. Cramer, R., Gennaro, R., Schoenmakers, B.: A secure and optimally efficient multi-authority election scheme. In: Fumy, W. (ed.) EUROCRYPT'97. LNCS, vol. 1233, pp. 103–118. Springer, Berlin, Heidelberg (May 1997). https://doi.org/10.1007/3-540-69053-0_9
4. ElGamal, T.: A public key cryptosystem and a signature scheme based on discrete logarithms. IEEE Transactions on Information Theory **31**(4), 469–472 (1985). https://doi.org/10.1109/TIT.1985.1057074
5. Fouque, P.A., Poupard, G., Stern, J.: Sharing decryption in the context of voting or lotteries. In: Frankel, Y. (ed.) FC 2000. LNCS, vol. 1962, pp. 90–104. Springer, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-45472-1_7
6. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 305–326. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49896-5_11
7. Shamir, A.: How to share a secret. Communications of the ACM **22**(11), 612–613 (Nov 1979). https://doi.org/10.1145/359168.359176